



UNIVERSIDAD DE CARABOBO
FACULTAD DE CIENCIAS Y TECNOLOGÍA
ESCUELA DE COMPUTACIÓN
CAMPUS BÁRBULA
ARQUITECTURA DEL COMPUTADOR

Práctica de Laboratorio 4: Gestión de Entrada/Salida mediante Interrupciones y MMIO en MIPS32

Daniel Gutierrez CI: 32047059
Gabriel Diaz CI: 31161550

Bárbula, Octubre de 2026

1. Ciclo de una interrupción de hardware

El ciclo de atención a una interrupción sigue un flujo estricto que pausa temporalmente el programa principal para atender al periférico:

1. **Ocurrencia del evento:** Un dispositivo (ej. teclado) envía una señal eléctrica de petición al procesador.
2. **Suspensión y Respaldo de Hardware:** El procesador termina la instrucción actual. Automáticamente guarda la dirección de la siguiente instrucción en el registro EPC (`Exception Program Counter`) y cambia el registro `Status` a modo kernel.
3. **Salto a la ISR:** El flujo salta incondicionalmente a la dirección fija de la Rutina de Servicio de Interrupción (`0x80000180` en MARS).
4. **Respaldo de Software (Contexto):** La ISR guarda manualmente en la memoria RAM los registros generales (como `$t0, $t1`) que utilizará.
5. **Ejecución:** La ISR lee el registro `Cause` para identificar quién interrumpió y ejecuta las acciones pertinentes.
6. **Restauración y Retorno:** La ISR restaura los registros generales desde la RAM y ejecuta `eret` (Exception Return), lo que restaura el modo usuario y el PC desde el EPC, reanudando el programa principal exactamente donde se quedó.

2. Diferencias entre Sondeo (Polling) e Interrupciones

- **Sondeo (Polling):** El procesador entra en un ciclo constante (busy-waiting) preguntándole repetidamente al periférico si tiene datos listos. Esto acapara el 100 % de la CPU, impidiendo que ejecute otras tareas útiles mientras espera.
- **Interrupciones:** El procesador sigue ejecutando el programa principal libremente. Es el dispositivo periférico el que "toca la puerta"(envía una señal asíncrona) únicamente cuando tiene un dato listo para ser procesado, avisándole a la CPU para que lo atienda de forma puntual.

3. Ventajas del uso de interrupciones en términos de uso del procesador

La principal ventaja es la **maximización de la eficiencia y el multiprocesamiento**. Al eliminar los bucles de espera (sondeo), el procesador puede dedicar todos sus ciclos de reloj a avanzar con la lógica principal del programa u otros procesos del sistema operativo. Además, reduce drásticamente el consumo energético, ya que la CPU no trabaja innecesariamente esperando un evento de entrada/salida que podría tardar segundos o minutos en ocurrir.

4. Registros especiales en MIPS32 para gestionar interrupciones

Estos registros pertenecen al Coprocesador 0 (CP0):

- **Registro 12 (Status):** Controla la habilitación global e individual de las interrupciones. Determina si el procesador está .esçuchando.º ignorando las señales externas.
- **Registro 13 (Cause):** Almacena un código numérico que identifica el origen exacto de la excepción o interrupción (qué dispositivo la generó).
- **Registro 14 (EPC - Exception Program Counter):** Guarda la dirección de memoria exacta de la instrucción que fue interrumpida en el programa principal, para poder regresar a ella.

5. ¿Por qué es necesario guardar el contexto (registros) en una rutina de servicio?

Porque la interrupción es impredecible y ocurre en medio del programa principal. Si el programa principal estaba usando el registro `$t0` para calcular un dato crítico, y la ISR utiliza ese mismo `$t0` sin respaldarlo previamente, el dato original se sobrescribirá y se perderá para siempre. Al guardar el contexto (respaldar en RAM y luego restaurar), garantizamos que la ISR sea ïnvisiblez no corrompa el estado lógico de la ejecución general.

6. Momentos en que pueden generarse excepciones en un sistema MIPS32

a) Cuatro situaciones que generan excepciones:

1. Desbordamiento aritmético (Overflow) al ejecutar sumas o multiplicaciones con signo.
2. Intento de ejecutar una instrucción con un código de operación (Opcode) desconocido o inválido.
3. Fallo de alineación de memoria (intentar cargar una palabra (`lw`) desde una dirección que no es múltiplo de 4).
4. Ejecución de la instrucción `syscall`, que provoca una excepción de software planificada para pedir servicios al sistema operativo.

b) Etapas del pipeline y excepciones:

- **Etapa IF (Instruction Fetch):** Fallo de página (Page Fault) o violación de acceso, al intentar buscar una instrucción en una memoria no permitida.
- **Etapa ID (Instruction Decode):** Instrucción indefinida o reservada.
- **Etapa EX (Execute):** Desbordamiento aritmético en la ALU.
- **Etapa MEM (Memory Access):** Error de dirección de memoria (desalineación) al intentar hacer lecturas/escrituras.

7. Estrategias de tratamiento de excepciones e interrupciones

a) Diferencias (Síncronas vs Asíncronas): Las **Excepciones** son *síncronas*; son generadas internamente por la instrucción que se está ejecutando en ese preciso momento (ej. división por cero). Las **Interrupciones** son *asíncronas*; provienen de dispositivos físicos externos (como un teclado o un temporizador) y no tienen relación con la instrucción que la CPU está procesando en ese instante.

b) Estrategias de tratamiento y redirección:

- **Interrupciones no vectorizadas (usadas en MARS):** Todas las excepciones dirigen el hardware a una única dirección fija (`0x80000180`). La ISR debe consultar el registro `Cause` para descubrir qué pasó y actuar.
- **Interrupciones vectorizadas:** El hardware salta a una dirección de memoria diferente dependiendo de la causa de la interrupción, eliminando la necesidad de interrogar por software.

El hardware redirige la ejecución alterando el `PC` forzosamente hacia `0x80000180`. La función del registro `EPC` es ser la çaja fuerte"que resguarda la dirección del PC original antes del salto, sirviendo de ancla para retornar.

8. Habilitación de interrupciones en dispositivos y procesador

a) Cómo se habilitan:

- **Teclado:** Escribiendo un 1 en el Bit 1 del registro de control de entrada (MMIO) ubicado en la dirección `0xFFFF0000`.
- **Pantalla:** Escribiendo un 1 en el Bit 1 del registro de control de salida (MMIO) ubicado en `0xFFFF0008`.
- **Procesador:** Copiando el registro `Status` (Reg 12) a un registro general, aplicando una máscara OR (`0x0101`) para encender el bit de interrupción global (IE, bit 0) y los bits de máscara específicos, y regresándolo al Coprocesador 0 con `mtc0`.

b) ¿Qué pasaría si habilitamos dispositivos pero no procesador? El teclado o la pantalla detectarán los eventos y enviarán señales eléctricas hacia el procesador, pero este las ignorará por completo. Actuaría como un procesador "sordo", ya que al tener el bit IE apagado en el `Status`, el hardware tiene prohibido suspender la ejecución actual.

9. Procesamiento de interrupciones

a) Ocurrencia de una interrupción de reloj:

1. *El hardware interviene:* Detecta la señal del temporizador, pasa a modo Kernel, apaga temporalmente las nuevas interrupciones globales, guarda el PC actual en el `EPC` y fuerza el salto a `0x80000180`.

2. *El software interviene (Rutina ISR):* Desactiva temporalmente el uso del registro `$at`, respalda registros generales (como `$t0, $t1`) en variables en `.data`, consulta el `Cause`, confirma que es el reloj, y actualiza el contador de tiempo interno.

3. *Finalización:* El software restaura los registros `$t` y el `$at`, y llama a la instrucción `eret`. El hardware toma el valor de `EPC`, lo devuelve al PC y cambia de vuelta a modo Usuario.

b) Importancia de guardar el contexto (registros y EPC): Es fundamental porque el sistema operativo maneja múltiples procesos. Si una interrupción altera los registros generales o sobreescribe los registros del Coprocesador sin respaldarlos, el hilo de ejecución original se corrompe, causando que la aplicación crashee al reanudarse por calcular sobre datos alterados.

10. Interrupciones de reloj y control de ejecución

a) Usos del temporizador:

- **Bucle infinito:** El sistema operativo usa el reloj para generar interrupciones periódicas ("ticks"). Cada tick le quita el control momentáneo a la aplicación (Preemption). Si el SO detecta que un proceso ha estado atrapado consumiendo demasiados ciclos sin avanzar, lo fuerza a terminar, evitando que el computador se congele.

- **Finalizar programas por tiempo límite:** Similar al anterior, se configura un contador en la interrupción. Si la cantidad de interrupciones de reloj excede el equivalente al "timeout"permitido para ese programa, el SO descarta el hilo de ejecución y libera la memoria.

b) Si finaliza antes del temporizador: Si el programa termina exitosamente por su cuenta, hace un llamado al sistema (syscall) de salida. El Sistema Operativo toma el control, desvincula o cancela el contador asociado a ese proceso, y reasigna los ciclos de reloj sobrantes al siguiente programa en la cola, maximizando la eficiencia.

11. Análisis y Discusión de los Resultados

La práctica demostró que el diseño de sistemas interactivos modernos depende totalmente de las interrupciones mediante Entrada/Salida Mapeada en Memoria (MMIO). A través de los ejercicios del buffer y del semáforo, se comprobó que el enfoque tradicional de sondeo (polling) es obsoleto y perjudicial para el hardware, al mantener al procesador en bucles inútiles. La implementación manual del almacenamiento de contexto y el manejo de los registros de Coprocesador 0 ilustró a bajo nivel los principios del diseño de Sistemas Operativos, validando que separar la recolección asíncrona de datos de la lógica central del programa es el mecanismo definitivo para lograr la multitarea real en arquitectura computacional.